

Automation과 SDK

에이전트를 코드와 파이프라인에 통합

One agent for everywhere you code - CLI, IDE, App, Web

Codex Deep Dive Workshop (2-Day) - Module 9 / Automation and SDK

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



MODULE 09

Automation과 SDK

에이전트를 코드와 파이프라인에 통합



이 모듈의 목표

이 모듈에서 얻어갈 네 가지

GOAL

이 모듈을 마치면 exec를 스크립트에 결합하고, SDK로 앱에 에이전트를 임베드하며, GitHub Action으로 CI를 자동화하고, MCP 서버를 직접 만들 수 있습니다.

EXEC

스크립트 결합

종료 코드와 JSON으로 파이프라인을 제어합니다

SDK

앱 임베드

코드에서 에이전트를 라이브러리로 호출합니다

ACTION

CI 자동화

GitHub Action으로 워크플로우에 통합합니다

MCP

서버 구축

내부 시스템을 MCP 서버로 노출합니다

65 min

강의 45 + 랩 20

exec

스크립트 결합

SDK

프로그래밍 호출

Lab 8

CI 통합 실습

자동화 개요

프로그래밍 방식의 Codex

EXEC

비대화형 실행

codex exec로 스크립트에서 한 번에 실행합니다

SDK

SDK

코드에서 Codex를 라이브러리로 호출합니다

ACTION

GitHub Action

CI 워크플로우에 에이전트를 통합합니다

AUTOMATION VALUE 자동화하면 사람이 매번 지시하지 않아도 정해진 작업이 일관되게 실행됩니다.
대량 작업과 반복 작업에 특히 강력합니다.

SPECTRUM

스펙트럼: exec(스크립트) SDK(앱 임베드) Action(CI)로 갈수록 통제 수준 상승

exec

가장 단순

SDK

세밀한 제어

Action

CI 통합

일관성

반복 작업 강점

비대화형 실행 심화

codex exec 스크립팅

Terminal - scripting

```
# 종료 코드와 출력으로 파이프라인 제어
$ codex exec "Run tests and fix failures" --cd ./service
$ echo $?                # 0=성공, 비0=실패

# JSON 출력 모드로 결과를 구조적으로 파싱
$ codex exec --json "Summarize changes" | jq .summary
```

SCRIPTABLE exec는 종료 코드, 표준 출력, JSON 모드를 제공해 셸 스크립트와 파이프라인에 자연스럽게 결합됩니다. 성공과 실패를 코드로 판정할 수 있는 것이 핵심입니다.

\$?

종료 코드 판정

--json

구조적 출력

| jq

파이프 파싱

--cd

작업 디렉토리

SDK Overview

코드로 Codex 호출

CODEx SDK

SDK는 애플리케이션 코드에서 Codex 에이전트를 프로그래밍 방식으로 실행하게 합니다.

커스텀 도구와 자동화 시스템을 구축할 수 있습니다.

EMBED

임베드

내 앱에 에이전트 기능을 내장합니다

CONTROL

제어

실행, 승인, 결과를 코드로 제어합니다

STREAM

스트리밍

진행 상황을 실시간으로 수신합니다

CUSTOM

커스텀

도구와 워크플로우를 직접 정의합니다

Embed

앱에 내장

Control

코드 제어

Stream

실시간 수신

Custom

워크플로우 정의

SDK 사용 예시

TypeScript 호출

TypeScript - codex-sdk (개념)

```
// Codex SDK 호출 예시
import { Codex } from "@openai/codex-sdk"
const codex = new Codex({ apiKey: process.env.OPENAI_API_KEY })

const result = await codex.run({
  task: "Add unit tests for the cart module"
  cwd: "./shop", approvalPolicy: "never" // 격리 환경만
})
console.log(result.summary)
```

CONTROL IN CODE task, cwd, approvalPolicy를 코드로 지정하고 결과를 객체로 받습니다.
never 승인은 반드시 격리 환경에서만 사용하십시오.

run()

작업 실행

task

지시 전달

result

구조적 결과

never

격리 환경 전용

App Server

에이전트를 서비스로

APP SERVER

App Server 모드로 Codex를 장기 실행 서비스로 띄워, 여러 요청을 받아 처리하는 백엔드로 활용할 수 있습니다.

1 Start

서버 기동

2 Request

작업 요청 수신

3 Execute

에이전트 실행

4 Respond

결과 반환

WHEN TO USE 매번 프로세스를 새로 띄우는 대신, 다수의 요청을 받는 백엔드가 필요할 때 적합합니다.
사내 자동화 플랫폼이나 봇 서비스의 코어로 활용합니다.

Long-run

장기 실행

Multi-req

다중 요청

Backend

서비스화

Scalable

확장 가능

GitHub Action

CI에 에이전트 통합

.github/workflows/codex.yml

```
# 이슈 라벨링 시 Codex 작업 트리거 (개념 예시)
name: Codex Task
on: { issues: { types: [labeled] } }
jobs:
  run-codex:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: openai/codex-action@v1
```

EVENT-DRIVEN 이슈 라벨, PR 생성, 스케줄 같은 GitHub 이벤트가 워크플로우를 트리거합니다.
시크릿은 with가 아니라 GitHub Secrets로 주입하십시오.

on:
이벤트 트리거

checkout
코드 체크아웃

codex-action
공식 액션

Secrets
안전한 키 주입

Action 활용 패턴

CI 자동화 패턴과 주의점

PATTERNS

자동화 패턴

- 이슈 라벨링으로 작업 자동 트리거
- PR에 자동 리뷰 코멘트 생성
- 야간 배치로 의존성 업데이트
- 실패한 CI를 자동 분석하고 수정 제안

OPERATIONS

운영 주의점

- 시크릿은 GitHub Secrets로 안전하게 주입
- 네트워크와 권한을 최소로 제한
- 에이전트 변경도 리뷰와 CI 게이트 통과
- 실행 로그를 보존해 감사 가능하게

SAME PRINCIPLE Module 8의 보안 원칙이 그대로 적용됩니다.
CI의 에이전트는 사람 감독이 없으므로 격리와 최소 권한이 더욱 중요합니다.

Label

이슈 트리거

Review

PR 코멘트

Nightly

야간 배치

Secrets

안전 주입

MCP 서버 구축

내부 연동과 멀티 에이전트

BUILD MCP

MCP 서버 만들기

- 내부 도구의 tools를 표준으로 정의
- 핸들러를 구현해 동작 연결
- MCP 프로토콜로 노출
- config에 등록해 권한 통제

CODEX AS SERVER

Codex를 MCP 서버로

- codex mcp-server로 Codex를 도구화
- 계획 에이전트가 작업을 분해
- 여러 Codex 인스턴스에 하위 작업 위임
- 인스턴스별 샌드박스로 병렬 실행

ORCHESTRATION Codex 자체를 MCP 서버로 띄우면 다른 에이전트가 Codex를 도구처럼 호출합니다.
계획 에이전트가 큰 작업을 쪼개 여러 Codex에 위임하는 멀티 에이전트 구성이 가능합니다.

Define

도구 정의

Expose

표준 노출

mcp-server

Codex 도구화

Parallel

인스턴스 병렬

자동화 아키텍처 예시

레퍼런스 아키텍처

1 Trigger

이슈, 스케줄, 이벤트

2 Codex

에이전트 실행

3 Verify

테스트, 검토

4 Deliver

PR, 알림

COMPONENTS

구성 요소

- 트리거, 이슈 라벨이나 cron 스케줄, 웹훅 이벤트
- 실행, 격리 환경에서 SDK 또는 Action으로 Codex 구동
- 검증, 테스트와 정적 분석에 필요 시 사람 승인 / 전달, PR 생성과 Slack 알림으로 결과 공유

BLUEPRINT 트리거, 실행, 검증, 전달의 네 단계가 모든 자동화의 뼈대입니다. Lab B에서 이 아키텍처를 직접 구현합니다.

Trigger

이벤트 시작

Execute

격리 실행

Verify

테스트, 승인

Deliver

PR, 알림

Module 9 요약

핵심 정리

ONE LINE

exec는 종료 코드와 JSON으로 스크립트에, SDK는 코드로 앱에, Action은 이벤트로 CI에 에이전트를 통합하며, 모두 트리거-실행-검증-전달 패턴을 따릅니다.

EXEC/SDK

프로그래밍 호출

스크립트 결합과 앱 임베드로
제어합니다

ACTION

CI 통합

이벤트 기반으로 워크플로우를
자동화합니다

MCP

내부 연동

서버 구축과 멀티 에이전트로
확장합니다

PATTERN

자동화 패턴

트리거, 실행, 검증, 전달의 뼈
대입니다

Scriptable

exec 결합

Embed

SDK 임베드

Event

Action 트리거

Lab B

다음 종합 실습

Mini Lab 8 (20분)

exec를 CI 파이프라인에 넣기

MISSION

로컬에서 exec 동작을 확인하고, GitHub Actions 워크플로우를 만들어 이슈 라벨로 트리거되게 한 뒤, 시크릿을 안전하게 주입합니다.

Terminal - mini lab 8

```
# Step 1. 로컬에서 동작 확인
$ codex exec "Run lint and fix violations"; echo $?
# Step 2. 워크플로우 파일 생성 (앞 슬라이드 예시 참고)
$ ${EDITOR:-vi} .github/workflows/codex.yml
# Step 3. 시크릿 등록 후 푸시, 라벨로 트리거
$ git commit -am "ci: add codex task" && git push
```

주의 / CHECKPOINT CI의 에이전트는 승인이 느슨하므로 반드시 격리 환경에서 쓰십시오. exec 종료 코드 0, 워크플로우 노출, 라벨 트리거, Secrets로만 키 주입을 확인하면 통과입니다.

20 min

제한 시간

Local

먼저 동작 확인

Label

이슈 라벨 트리거

Secrets

안전한 키 주입

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

